

EvidenceRank

Intelligent Candidate Discovery & Ranking — Redrob Hackathon

Team Name: Hunter Association

Problem Statement: Rank the top 100 of 100,000 candidates against a Senior AI Engineer JD by genuine fit (not keywords), within a CPU-only, ≤ 5 -min, no-network budget.

Team Leader Name: Shubham Mittal

Solution Overview

What is your proposed solution? What differentiates it from traditional matching?

- **Proposed solution**

- EvidenceRank: a deterministic, dependency-free ranker that scores candidates on grounded evidence of fit. Each candidate becomes (requirement, evidence, satisfaction) triples derived from a declarative model of the JD.

- **What differentiates it**

- Evidence over keywords — a skill counts only if backed by endorsements × duration × assessment, defeating keyword-stuffers.
- Bidirectional behavioral signals — down-weights stale / unresponsive candidates who can't actually be hired, exactly as the JD instructs.
- Honeypot consistency gate — internally-impossible profiles forced out (0 in the top 100).
- Every design choice kept or rejected by an offline eval harness, never by intuition.
- No LLM during ranking → fully reproducible, CPU-only, ~12s for 100K candidates.

JD Understanding & Candidate Evaluation

Key requirements · most important signals · evaluating fit beyond keywords

- **Key requirements extracted (declarative requirements.json, each quoting the JD)**

- Must-have: embeddings/retrieval, vector DB / hybrid search, shipped ranking-search-recsys at product companies, strong Python, evaluation frameworks (NDCG/MRR/MAP), 5–9 yrs, Noida/Pune/India.
- Anti-patterns: keyword stuffers, consulting-only, research-only, CV/speech-only, title-chasers.

- **Most important signals**

- Builder identity (title), trust-weighted core skills, career evidence of shipped systems, and behavioral availability (recruiter response rate, recency, open-to-work).

- **Beyond keyword matching**

- Reasons about the gap between what the JD says vs. means; trust-weights claims; weighs whether the candidate is actually hireable; detects subtly-impossible honeypots.

Ranking Methodology

How candidates are retrieved, scored & ranked · models/heuristics · signal fusion

- **Retrieve → score → rank**

- Single streaming pass over 100K, top-K min-heap, deterministic tie-break (rounded score, then candidate_id ascending — matches the validator).

- **Scoring model**

- final = base × trap_penalties × behavioral_modifier ; honeypots forced ≈ 0 .

- base = weighted sum of six grounded components — builder 0.22, skills 0.20, career 0.26, experience 0.16, location 0.10, eval 0.06.

- **Heuristics (explainable, no ML black box)**

- Skill trust-weighting; floored temporal decay on career recency; floored behavioral multiplier; multiplicative trap penalties. Weights tuned against a hand-labeled gold set.

Explainability & Data Validation

Explaining decisions · preventing hallucination · handling suspicious profiles

- **How decisions are explained**

- Every score is a list of (requirement, evidence, satisfaction) triples, each tied to a verbatim JD quote; the reasoning cites named skills, the company of the strongest retrieval role, and eval signals.

- **Preventing hallucination / unsupported justifications**

- Reasoning is generated deterministically from actual profile fields — no LLM, so every claim is grounded; nothing is invented.

- **Inconsistent / low-quality / suspicious profiles**

- Honeypot internal-consistency checks (job duration > its window, experience > career span, ≥ 5 'expert' skills with 0 months used) push them out.
 - Keyword-stuffer title gate + skill trust-weighting discount unbacked claims.

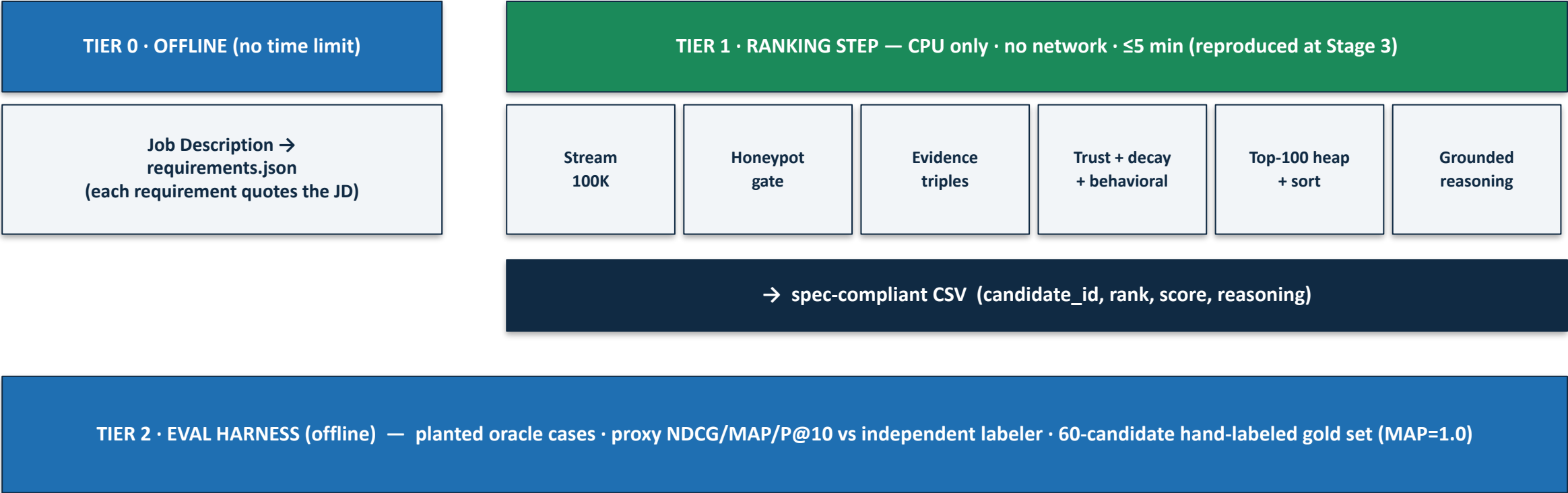
End-to-End Workflow

From JD input to ranked candidate output

- 1. JD → declarative requirements model (offline, one-time).
- 2. Stream candidates.jsonl (100K, line by line).
- 3. Honeypot consistency gate → impossible profiles forced near-zero.
- 4. Evidence matching → (requirement, evidence, satisfaction) triples per candidate.
- 5. Trust-weight skills · temporal decay · behavioral multiplier · trap penalties.
- 6. Combine → final score; maintain a top-100 heap.
- 7. Deterministic sort → grounded 1–2 sentence reasoning per candidate.
- 8. Spec-compliant CSV → organizer validator → 'Submission is valid.'

System Architecture

Three tiers — offline model · CPU-only ranking step · evaluation harness



- Stdlib-only (json, heapq, csv, datetime) → reproduces in any sandbox, no dependency risk.
- No hosted LLM, no GPU, no network anywhere in the ranking path.

Results & Performance

Ranking quality · meeting the runtime & compute constraints

- **Ranking quality (validated, not asserted)**

- Hand-labeled gold set (60 candidates): MAP = 1.000, NDCG@10 = 0.96 — the 3 genuine ML candidates ranked at the very top, above ~10 keyword-stuffer traps.
- Planted regression suite: 8/8 (perfect $\gg$ stale $\gg$ stuffer $\approx$ honeypot $\approx$ 0).
- 0 honeypots and 0 keyword-stuffers in the top 100 (honeypot DQ threshold is 10%).

- **Compute-constraint compliance (spec §3)**

- ~12 s for 100K on a laptop CPU (budget: 5 min) · streams line-by-line ($\ll$ 16 GB) · CPU-only · no network · zero third-party dependencies → reproduces anywhere.

Technologies Used

What was used, and why it was selected

- **Python 3.9+ standard library only**

- json, heapq, csv, datetime, collections — chosen for guaranteed Stage-3 reproducibility (no dependency or version risk in the sandbox).

- **Rule-based, explainable scoring (no ML black box)**

- The challenge bans per-candidate LLM calls and rewards CPU-scalable systems; explicit heuristics are fast, defensible at interview, and auditable.

- **Streamlit — hosted sandbox UI only (not in the ranking path).**

- **Claude (Anthropic) — pair-programmer for design & code review; no LLM is called during ranking.**

Submission Assets

GitHub · demo video · sandbox · ranked output

- **GitHub repository:** <https://github.com/Shubham-33/evidencerank-redrob>
- **Hosted sandbox / demo:** <https://evidencerank-redrob-dyblpprr79cibcoexhac5t.streamlit.app/>
- **Live demo:** the hosted Streamlit sandbox (link above) — runs the ranker on any sample.
- **Walkthrough:** step-by-step usage & reproduce steps in README.md.
- **Ranked submission:** submission.csv (passes organizer validate_submission.py)
- **Reproduce command:**
 - python execution/rank.py --candidates ./candidates.jsonl --out ./submission.csv
- **Methodology & every eval-gated decision documented in PLAN.md + directives/.**

EvidenceRank — evidence over keywords.

Reproducible · CPU-only · defensible line-by-line. Thank you.